```
# %%
import numpy as np

# %%
# Q1: Create a NumPy 1D array with 5 elements and perform basic operations

arr = np.array([1, 2, 3, 4, 5])
print("Original array:", arr)

# a) Addition of 2 in all elements
add_result = arr + 2
print("After adding 2:", add_result)

# b) Multiply 3 with all elements
mul_result = arr * 3
print("After multiplying by 3:", mul_result)

# c) Divide every element by 2
div_result = arr / 2
print("After dividing by 2:", div_result)

# %%
# Q2 a) Reverse the NumPy array

arr = np.array([1, 2, 3, 6, 4, 5])
reversed_arr = arr[::-1]
print("Original array:", arr)
print("Reversed array:", reversed_arr)

# %%
# Q2 b) Find the most frequent value and its indices

# i. For array x
x = np.array([1, 2, 3, 4, 5, 1, 2, 1, 1, 1])
values, counts = np.unique(x, return_counts=True)
most_frequent_x = values[np.argmax(counts)]
indices_x = np.where(x == most_frequent_x)[0]
print("Array x:", x)
print("Most frequent value:", most_frequent_x)
print("Indices:", indices_x)

# ii. For array y
y = np.array([1, 1, 1, 2, 3, 4, 2, 4, 3, 3])
```

```
values, counts = np.unique(y, return_counts=True)
most_frequent_y = values[np.argmax(counts)]
indices_y = np.where(y == most_frequent_y)[0]
print("\nArray y:", y)
print("Most frequent value:", most_frequent_y)
print("Indices:", indices_y)

# %%
# Q3: Access elements from a 2-D array using row and column indices

arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print("2-D array:\n", arr)

# a) Access 1st row, 2nd column
a = arr[0, 1]
print("1st row, 2nd column:", a)

# b) Access 3rd row, 1st column
b = arr[2, 0]
print("3rd row, 1st column:", b)

# %%
# Q4: 1-D array named Jaskaran with 25 evenly spaced numbers from 10 to 100

Jaskaran = np.linspace(10, 100, 25)
print("Array Jaskaran:\n", Jaskaran)

print("\nDimensions:", Jaskaran.ndim)
print("Shape:", Jaskaran.shape)
print("Total elements:", Jaskaran.size)
print("Data type:", Jaskaran.dtype)
print("Total bytes consumed:", Jaskaran.nbytes)

# Transpose using reshape (since it's 1-D, reshape can make it a column vector)
transpose_reshape = Jaskaran.reshape(25, 1)
print("\nTranspose using reshape (25,1):\n", transpose_reshape)

# Using T attribute
transpose_T = Jaskaran.T
print("\nUsing T attribute:\n", transpose_T)
print("\nNote: T attribute has no effect on a 1-D array (shape stays the same).")
print("To get a real column/row transpose effect on a 1-D array, reshape() must b

# %%
# Q5: 2-D Array named ucs420_Jaskaran with 3 rows and 4 columns
```

```
ucs420_Jaskaran = np.array([[10, 20, 30, 40],
                            [50, 60, 70, 80],
                            [90, 15, 20, 35]])
print("ucs420_Jaskaran:\n", ucs420_Jaskaran)

print("\nMean:", np.mean(ucs420_Jaskaran))
print("Median:", np.median(ucs420_Jaskaran))
print("Max:", np.max(ucs420_Jaskaran))
print("Min:", np.min(ucs420_Jaskaran))
print("Unique elements:", np.unique(ucs420_Jaskaran))

# Reshape to 4 rows and 3 columns
reshaped_ucs420_Jaskaran = ucs420_Jaskaran.reshape(4, 3)
print("\nreshaped_ucs420_Jaskaran (4x3):\n", reshaped_ucs420_Jaskaran)

# Resize to 2 rows and 3 columns
resized_ucs420_Jaskaran = np.resize(ucs420_Jaskaran, (2, 3))
print("\nresized_ucs420_Jaskaran (2x3):\n", resized_ucs420_Jaskaran)
```